

Elbow method

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris['feature_names'])
#print(X)
data = X[['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)']]

sse = {}
for k in range(1, 10):
    kmeans = KMeans(n_clusters=k, max_iter=300).fit(data)
    data["clusters"] = kmeans.labels_
    #print(data["clusters"])
    sse[k] = kmeans.inertia_
plt.figure()
plt.plot(list(sse.keys()), list(sse.values()))
plt.xlabel("Number of cluster")
plt.ylabel("SSE")
plt.show()
```

Silhouette method

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
```

```
from sklearn.metrics import silhouette_score
```

```
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris['feature_names'])
#print(X)
data = X[['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)']]
```

```
for n_cluster in range(2, 11):
    kmeans = KMeans(n_clusters=n_cluster, n_init=20, max_iter=300).fit(data)
    label = kmeans.labels_
    sil_coeff = silhouette_score(data, label, metric='euclidean')

    print("For n_clusters={}, The Silhouette Coefficient is {}".format(n_cluster, sil_coeff))
```

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

iris = load_iris()
X = pd.DataFrame(iris.data,
columns=iris['feature_names'])
#print(X)
data = X[['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)']]

from sklearn import metrics
wcss=[]
res=[]
```

```
figsize=(9, 7)
clusters_range = range(2,15)

for k in clusters_range:
    km = KMeans(n_clusters=k, n_init=20, max_iter=300)
    km.fit(data)
    wcss.append(km.inertia_)
    res.append(metrics.silhouette_score(data,km.labels_))

_, ax1 = plt.subplots(figsize=figsize)
ax1.plot(clusters_range, wcss, label="wcss", color="red", marker=".")
ax1.set_xlabel("# clusters")
ax1.set_ylabel("wcss")
ax1.grid(True)

ax2 = plt.gca().twinx()
ax2.plot(clusters_range, res, label="silhouette score", color="blue", marker=".")
ax2.set_ylabel("silhouette score")

g1, gl1 = ax1.get_legend_handles_labels()
g2, gl2 = ax2.get_legend_handles_labels()
plt.legend(g1 + g2, gl1 + gl2)

plt.show()
```